

Capstone Project: Snowflake Data Warehouse & Analytics

End-to-End Retail Data Solution for Brazilian E-Commerce Dataset

Pratham Bajpai
8183612

Prepared By: Associate Engineer - Technology
Domain: Data Engineering & Analytics
Date: May 2026

SECTION 1: PROJECT OVERVIEW

This project focuses on designing and implementing a **scalable cloud-based data warehouse using Snowflake** to analyze Brazilian e-commerce data. The objective is to build a complete end-to-end data pipeline that ingests raw retail data, transforms it into structured formats, and enables advanced analytical insights through interactive dashboards.

The system follows a **modern layered architecture (Bronze, Silver, Gold)** to ensure data quality, scalability, and performance. Data is sourced from the publicly available **Olist Brazilian E-Commerce dataset**, stored in AWS S3, and ingested into Snowflake using an external stage.

The final output includes a **business-ready analytical model (Star Schema)** and an interactive **Streamlit dashboard** that provides insights into:

- Sales and revenue performance
- Customer behavior and segmentation
- Product and category performance
- Logistics and delivery efficiency
- Payment method trends

Additionally, advanced features such as **dynamic currency conversion, repeat customer analysis, and revenue distribution insights** are implemented to enhance decision-making capabilities.

SECTION 2 — PROJECT OBJECTIVES

The primary objectives of this project are:

- To analyze **sales performance and revenue trends over time**
- To generate **customer behavior insights**, including segmentation and repeat purchase patterns
- To track **product-level performance**, including top categories and revenue contribution
- To evaluate **operational efficiency**, including delivery time and shipping cost impact
- To provide a **scalable and optimized data warehouse solution** using Snowflake
- To develop an **interactive dashboard** for business stakeholders

SECTION 3 — ARCHITECTURE OVERVIEW

3.1 Architecture Description

The system follows a modern data pipeline architecture:

None

CSV Files → AWS S3 → Snowflake External Stage → Bronze → Silver → Gold → Streamlit Dashboard

- **Data Source (AWS S3):** Olist Brazilian E-Commerce Dataset contains Orders, Order Items, Customers, Products, Products Category, Payments, Reviews, Sellers, Geolocation CSV files.
- **Ingestion Layer (AWS S3):** Data was sourced as CSV files and staged in an AWS S3 bucket. Access was established via a Snowflake External Stage and Storage Integration, avoiding the security risks associated with internal staging.
- **Bronze Layer (Raw):** Acts as the landing zone. Data is loaded using the COPY INTO command with no modifications to ensure a "Single Source of Truth".
- **Silver Layer (Cleansed):** Data undergoes cleaning. VARCHAR timestamps are cast to TIMESTAMP formats, and categorical NULLs are handled (e.g., product categories coalesced to "Other").
- **Gold Layer (Business Model):** The final analytical layer structured as a Star Schema. Fact and Dimension tables are linked using MD5 surrogate keys for deterministic joins.
- **Analytics Layer:** Streamlit dashboard connected to Snowflake and provides interactive visual insight.

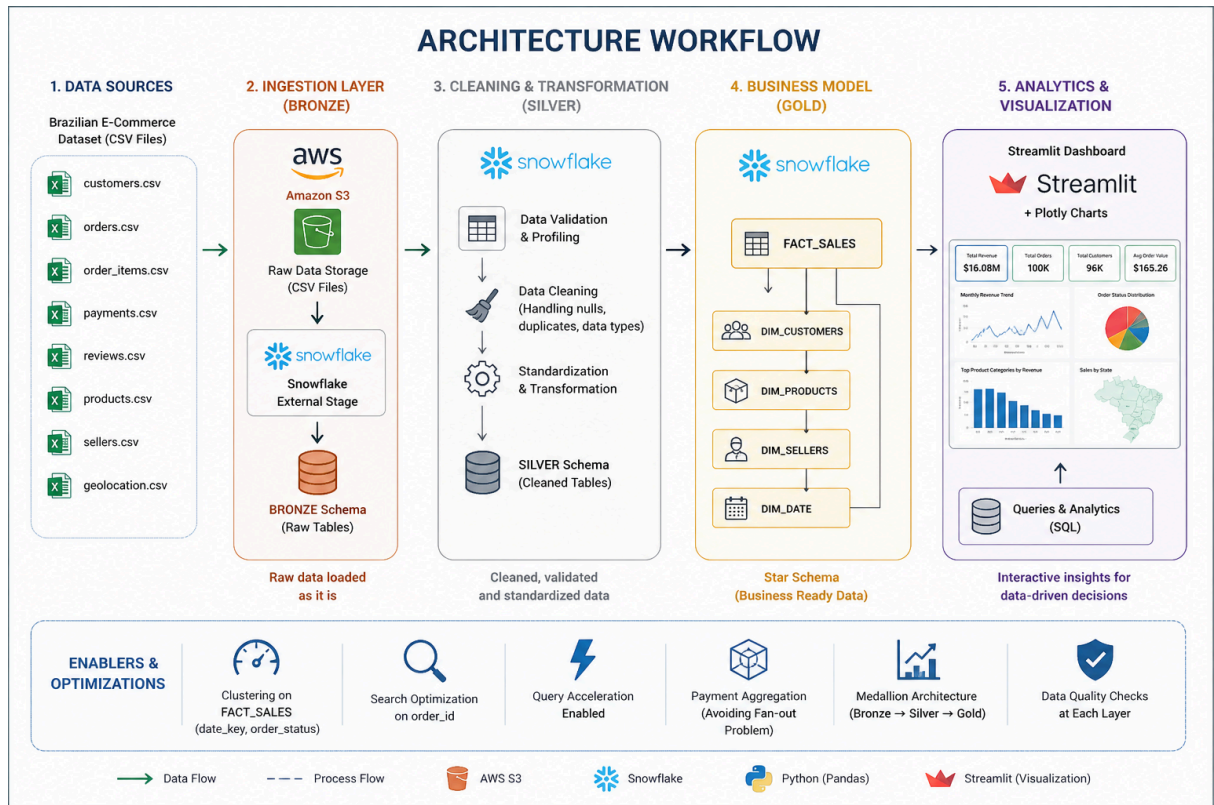


Fig 3.1 - Architecture Workflow

SECTION 4 — DATA INGESTION

4.1 Dataset Used

The project uses the **Olist Brazilian E-Commerce Dataset**, which contains real-world transactional data.

4.2 Files Used

- orders
- order_items
- customers
- payments
- reviews
- products
- products_category
- sellers
- geolocation

4.3 Ingestion Process

1. Upload CSV files to AWS S3
2. Create storage integration object in Snowflake
3. Create external stage in Snowflake
4. Define file format (CSV)
5. Load data using `COPY INTO`

4.4 Sample SQL

SQL

```
CREATE OR REPLACE STORAGE INTEGRATION s3_retail_brazilian_integration  
  
  TYPE = EXTERNAL_STAGE  
  
  STORAGE_PROVIDER = 'S3'  
  
  ENABLED = TRUE
```

```

    STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::000000000000:role/placeholder'

    STORAGE_ALLOWED_LOCATIONS = ('s3://snowflake-capstone-retail-data/');

DESC INTEGRATION s3_retail_brazilian_integration;

ALTER STORAGE INTEGRATION s3_retail_brazilian_integration

SET
                                STORAGE_AWS_ROLE_ARN
                                =
'arn:aws:iam::439855819219:role/Snowflake_Retail_Brazilian_S3_Access';


CREATE OR REPLACE STAGE retail_brazilian_external_stage

    URL = 's3://snowflake-capstone-retail-data/'

    STORAGE_INTEGRATION = s3_retail_brazilian_integration

    FILE_FORMAT = retail_brazilian_csv_format;

COPY INTO CUSTOMERS_RAW

FROM
@Retail_Brazilian_DB.Retail_Brazilian_SCHEMA.retail_brazilian_external_stage/csv_data
/olist_customers_dataset.csv

FILE_FORMAT = Retail_Brazilian_DB.Retail_Brazilian_SCHEMA.retail_brazilian_csv_format

ON_ERROR = 'CONTINUE';

```

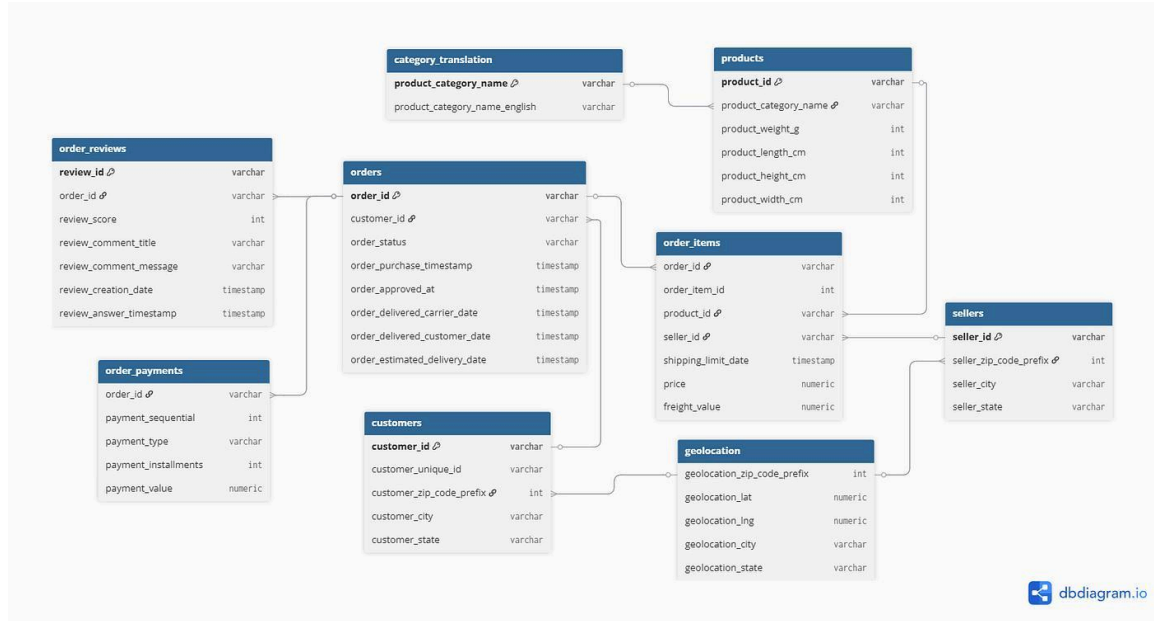


Fig 4.1 Database Design Schema

Table name	Rows	Datatype changes
olist_order_dataset → olist_order	99,441	VARCHAR(100) to TIMESTAMP: 'order_purchase_timestamp' 'order_approved_at' 'order_delivered_carrier_date' 'order_delivered_customer_date' 'order_estimated_delivery_date'
olist_order_items_dataset → olist_items	112,650	VARCHAR(100) to TIMESTAMP: 'shipping_limit_date'
olist_customers_dataset → olist_customers	99,441	INTEGER to VARCHAR(50): 'customer_zip_code_prefix'
olist_order_payment_dataset → olist_payment	103,886	
olist_products_dataset → olist_products	32,951	
olist_sells_dataset → olist_sells	3,095	INTEGER to VARCHAR(50): 'seller_zip_code_prefix'
olist_geolocation_dataset → olist_geolocation	1,000,163	INTEGER to VARCHAR(50): 'geolocation_zip_code_prefix'
olist_order_reviews_dataset → olist_reviews	100,000	See note
product_category_name	71	

Fig 4.2 Database Schema Information

SECTION 5 — DATA MODELING

5.1 Schema Definitions & Relationships

Central Fact Table: FACT_SALES

This table represents the grain of a single order item. It contains the quantitative metrics (facts) and the keys (foreign keys) that connect to the descriptive dimensions.

Primary Key: sales_key (MD5 Hash of order_id + product_id + seller_id).

Foreign Keys:

customer_key → Links to DIM_CUSTOMERS.

product_key → Links to DIM_PRODUCTS.

seller_key → Links to DIM_SELLERS.

date_key → Links to DIM_DATE.

Measures: price, freight_value, total_payment_value, order_status.

Dimension: DIM_CUSTOMERS

Contains descriptive attributes about the person making the purchase.

Primary Key: customer_key (MD5 of customer_unique_id).

Attributes: customer_city, customer_state, customer_zip_code.

Dimension: DIM_PRODUCTS

Contains metadata regarding the items sold.

Primary Key: product_key (MD5 of product_id).

Attributes: category_name, product_weight_g, product_length_cm, product_height_cm.

Dimension: DIM_SELLERS

Contains information about the merchants fulfilling the orders.

Primary Key: seller_key (MD5 of seller_id).

Attributes: seller_city, seller_state

Dimension: DIM_DATE

A generated time-dimension table to support time-series analysis (Monthly trends, Black Friday analysis).

Primary Key: date_key (Integer format YYYYMMDD).

Attributes: date, day, month, year, quarter, day_name, month_name.

5.2 Design Decisions for the ER Model

- **One-to-Many Relationships:** Every dimension has a 1:N relationship with the Fact table. One customer can have many sales, but one specific sale record belongs to exactly one customer.
- **Normalization vs. Performance:** We chose a Star Schema over a Snowflake Schema to minimize joins and optimize for the read-heavy workloads typical of the Streamlit dashboard.
- **Data Integrity:** The use of surrogate keys (MD5) ensures that even if the source order_id or customer_id changes format, our internal warehouse relationships remain stable.

5.3 Why this is the "Correct" Data Model:

- **Performance:** Analysts can join FACT_SALES to DIM_PRODUCTS easily without worrying about Portuguese-to-English translations.
- **Time Analysis:** The DIM_DATE table allows to perform "Sales and revenue trends over time" (Project Objective #4) by joining the purchase_date.
- **KPI Ready:** FACT_SALES table now has price, freight, and payment_value in one place for easy calculations.
- **Surrogate Keys:** Faster joins, Standard star schema practice, Needed for BI tools

5.4 Entity-Relationship Diagram (Star Schema)

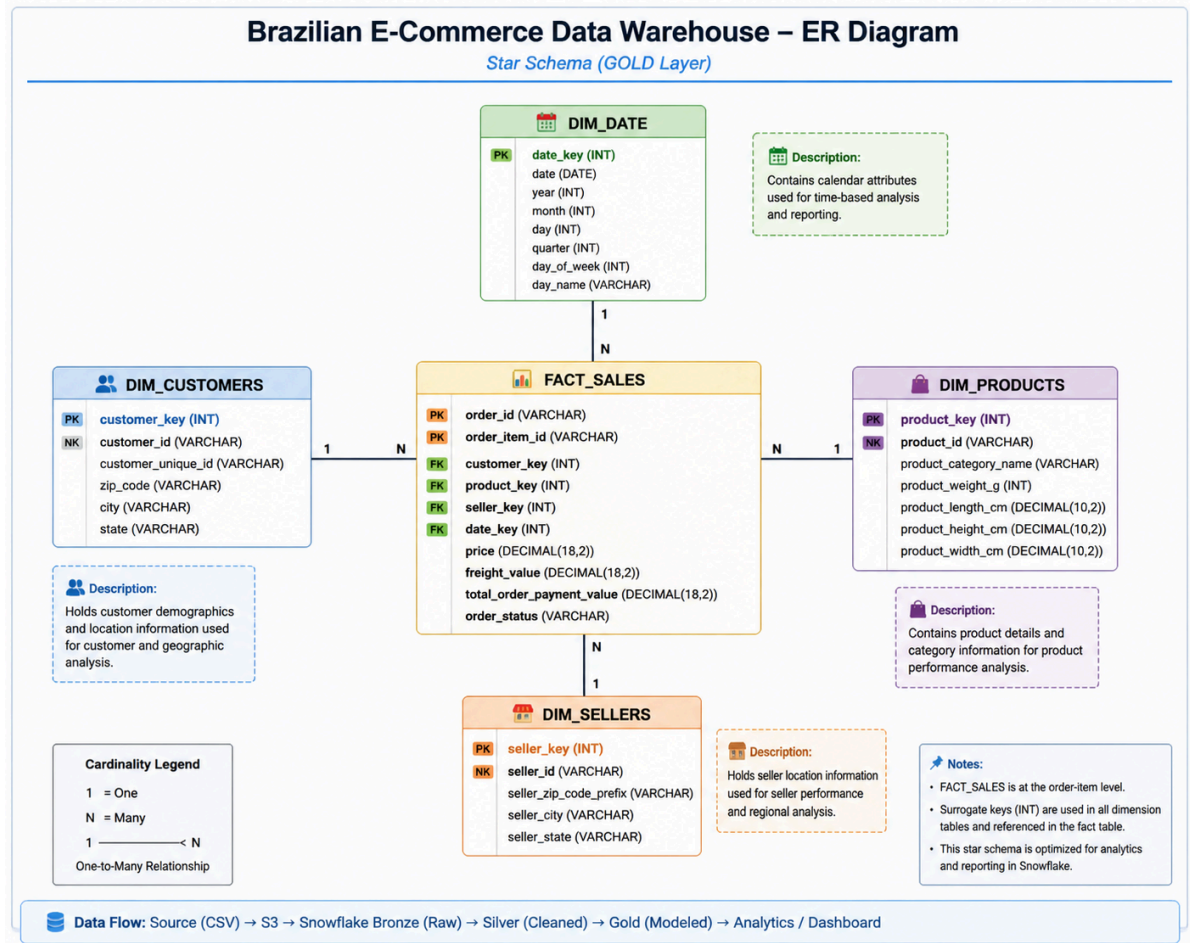


FIG 5.1 - Brazilian E-Commerce Data Warehouse ER DIAGRAM

SECTION 6 — ANALYTICAL QUERIES

To derive meaningful business insights, a set of structured SQL queries was developed and organized into logical groups based on business objectives.

7.1 Executive KPIs

Objective: To provide a high-level overview of business performance.

Query Description: This query calculates total orders, customers, revenue, and average order value.

```
SQL
SELECT
    COUNT(DISTINCT order_id) AS total_orders,
    COUNT(DISTINCT customer_key) AS total_customers,
    SUM(price + freight_value) AS total_revenue,
    ROUND(AVG(price + freight_value), 2) AS avg_order_value
FROM GOLD.FACT_SALES
WHERE order_status = 'delivered';
```

Insight:

- Provides overall business health
- Used in dashboard KPI tiles

7.2 Sales & Product Performance

- **Top Categories by Revenue**

Objective: Identify top-performing product categories.

```
SQL
SELECT
    dp.category,
```

```

COUNT(DISTINCT fs.order_id) AS total_orders,

SUM(fs.price) AS total_sales,

SUM(fs.price + fs.freight_value) AS total_revenue
FROM GOLD.FACT_SALES fs
JOIN GOLD.DIM_PRODUCTS dp
    ON fs.product_key = dp.product_key
WHERE fs.order_status = 'delivered'
GROUP BY dp.category
ORDER BY total_revenue DESC
LIMIT 10;

```

Insight:

- Helps identify high revenue categories
- Useful for inventory and marketing strategy

• Underperforming Categories (Shipping Impact)

Objective: Identify categories where shipping cost is high relative to price.

```

SQL
SELECT

    p.category,

    ROUND(SUM(f.price), 2) AS merchandise_revenue,

    ROUND(SUM(f.freight_value), 2) AS shipping_costs,

    ROUND((SUM(f.freight_value) / NULLIF(SUM(f.price), 0)) * 100, 2) AS
shipping_ratio_pct
FROM GOLD.FACT_SALES f
JOIN GOLD.DIM_PRODUCTS p ON f.product_key = p.product_key
WHERE f.order_status = 'delivered'
GROUP BY p.category
HAVING merchandise_revenue > 1000
ORDER BY shipping_ratio_pct DESC

```

```
LIMIT 10;
```

Insight:

- High shipping cost reduces profitability
- Helps optimize logistics pricing

- **Payment Method Analysis**

Objective: Understand customer payment preferences.

```
SQL
SELECT
    payment_type,
    COUNT(DISTINCT order_id) AS total_orders,
    ROUND(SUM(payment_value), 2) AS total_amount
FROM SILVER.PAYMENTS_CLEAN
GROUP BY payment_type
ORDER BY total_orders DESC;
```

Insight:

- Shows dominant payment methods
- Helps in financial planning

7.3 Customer & Market Analysis

- **Customer Segmentation**

Objective: Classify customers based on spending behavior.

```
SQL
SELECT
    customer_key,
    COUNT(DISTINCT order_id) AS total_orders,
```

```

SUM(price + freight_value) AS total_spent,
CASE
    WHEN SUM(price + freight_value) > 1000 THEN 'High Value'
    WHEN SUM(price + freight_value) > 500 THEN 'Medium Value'
    ELSE 'Low Value'
END AS segment
FROM GOLD.FACT_SALES
WHERE order_status = 'delivered'
GROUP BY customer_key;

```

Insight:

- Identifies high-value customers
- Supports targeted marketing

• Customer Distribution by Segment

```

SQL
SELECT
    CASE
        WHEN total_spent > 1000 THEN 'High Value'
        WHEN total_spent > 500 THEN 'Medium Value'
        ELSE 'Low Value'
    END AS segment,
    COUNT(customer_key) AS customer_count,
    ROUND(SUM(total_spent), 2) AS total_revenue
FROM (
    SELECT customer_key, SUM(price + freight_value) AS total_spent
    FROM GOLD.FACT_SALES
    WHERE order_status = 'delivered'
    GROUP BY 1
)

```

```
GROUP BY 1  
ORDER BY 1;
```

Insight: Shows contribution of each customer segment

- **Repeat Customer Analysis**

Objective: Analyze customer retention.

```
SQL  
SELECT  
    order_count,  
    COUNT(*) as customer_count  
FROM (  
    SELECT customer_key, COUNT(order_id) as order_count  
    FROM GOLD.FACT_SALES  
    WHERE order_status = 'delivered'  
    GROUP BY 1  
)  
GROUP BY 1  
ORDER BY 1;
```

Insight:

- Majority are one-time customers
- Indicates opportunity for retention strategies

- **Regional Performance**

```
SQL  
SELECT  
    c.state,  
    COUNT(DISTINCT f.order_id) AS total_orders,  
    SUM(f.price + f.freight_value) AS total_revenue,
```



```

        ROUND(SUM(f.price + f.freight_value) / COUNT(DISTINCT f.order_id), 2) AS
avg_order_value

FROM GOLD.FACT_SALES f

JOIN GOLD.DIM_CUSTOMERS c

    ON f.customer_key = c.customer_key

WHERE f.order_status = 'delivered'

GROUP BY c.state

ORDER BY total_revenue DESC;

```

Insight: Identifies high-performing regions

- **Monthly Revenue Trend**

```

SQL
SELECT

    dd.year,

    dd.month,

    SUM(fs.price + fs.freight_value) AS monthly_revenue,

    COUNT(DISTINCT fs.order_id) AS total_orders

FROM GOLD.FACT_SALES fs

JOIN GOLD.DIM_DATE dd

    ON fs.date_key = dd.date_key

WHERE fs.order_status = 'delivered'

GROUP BY dd.year, dd.month

ORDER BY dd.year, dd.month;

```

Insight: Tracks growth over time

- **Customer Satisfaction**

```

SQL
SELECT

```

```

        review_score,
        COUNT(*) AS total_reviews
FROM SILVER.REVIEWS_CLEAN
GROUP BY review_score
ORDER BY review_score;

```

Insight: Measures product/service satisfaction

7.4 Operations & Logistics

- **Order Status Distribution**

```

SQL
SELECT
    order_status,
    COUNT(DISTINCT order_id) AS total_orders
FROM GOLD.FACT_SALES
GROUP BY order_status
ORDER BY total_orders DESC;

```

Insight: Evaluates operational efficiency

- **Delivery Performance**

```

SQL
SELECT
    AVG(DATEDIFF(day, order_purchase_ts, order_delivered_customer_ts)) AS
    avg_delivery_days
FROM SILVER.ORDERS_CLEAN
WHERE order_status = 'delivered';

```

Insight: Measures logistics efficiency

- **Seller Performance**

```
SQL
SELECT

    ds.seller_city,

    ds.seller_state,

    COUNT(DISTINCT fs.seller_key) AS active_sellers,

    COUNT(DISTINCT fs.order_id) AS total_orders,

    SUM(fs.price) AS total_sales

FROM GOLD.FACT_SALES fs

JOIN GOLD.DIM_SELLERS ds

    ON fs.seller_key = ds.seller_key

WHERE fs.order_status = 'delivered'

GROUP BY ds.seller_city, ds.seller_state

ORDER BY total_sales DESC

LIMIT 10;
```

Insight: Identifies top-performing seller locations

SECTION 7 — PERFORMANCE OPTIMIZATION

To ensure efficient query execution and scalability, multiple performance optimization techniques were implemented in Snowflake. These optimizations focus on data modeling, storage optimization, and query efficiency.

7.1 Pre-Aggregation of Payment Data (Fan-out Problem Fix)

Objective: Prevent duplicate rows and incorrect aggregations.

Implementation

- Payments table contains multiple rows per order
- Aggregated payments using

```
SQL
WITH payments_agg AS (
    SELECT
        order_id,
        SUM(payment_value) AS total_payment_value
    FROM SILVER.PAYMENTS_CLEAN
    GROUP BY order_id
)
```

Benefit

- Eliminates row duplication
- Ensures accurate revenue calculations
- Improves query performance

7.2 Clustering of Fact Table

Objective: Improve query performance on large datasets.

Implementation

SQL

```
ALTER TABLE GOLD.FACT_SALES  
  
CLUSTER BY (date_key, order_status);
```

Benefit

- Faster filtering on date-based queries
- Efficient pruning of data
- Optimized time-series analysis

7.3 Search Optimization (Point Lookup Optimization)

Objective: Speed up selective queries.

Implementation

SQL

```
ALTER TABLE GOLD.FACT_SALES  
  
ADD SEARCH OPTIMIZATION ON EQUALITY(order_id);
```

Benefit

- Faster lookup for specific orders
- Improves performance for filtering queries

7.4 Query Acceleration (Warehouse Optimization)

Objective: Improve performance for large analytical queries.

Implementation

SQL

```
ALTER WAREHOUSE RETAIL_BRAZILIAN_WH  
  
SET ENABLE_QUERY_ACCELERATION = TRUE;
```

Benefit

- Reduces execution time for heavy queries
- Automatically optimizes query processing

7.5 Efficient Filtering Strategy

Objective: Reduce unnecessary data processing.

Implementation

- Applied filter:

SQL

```
WHERE order_status = 'delivered'
```

- Used in analytical queries and dashboard

Benefit

- Reduces data scan
- Improves performance
- Ensures relevant business analysis

7.6 Materialized View for KPI Optimization

Objective: Precompute frequently used metrics.

Implementation

SQL

```
CREATE OR REPLACE MATERIALIZED VIEW GOLD.MV_DAILY_REVENUE_KPI AS

SELECT

    date_key,

    order_status,

    SUM(price) AS total_sales,

    SUM(freight_value) AS total_freight,

    COUNT(order_id) AS order_count

FROM GOLD.FACT_SALES

GROUP BY date_key, order_status;
```

Benefit

- Faster dashboard loading
- Reduces computation time
- Improves KPI query performance

SECTION 8 — DASHBOARD IMPLEMENTATION

An interactive business intelligence dashboard was developed using **Streamlit** and **Plotly**, connected directly to Snowflake. The dashboard provides real-time analytical insights into sales, customers, operations, and overall business performance.

The dashboard is structured into multiple sections to support **data-driven decision making**.

8.1 Dynamic Multi-Currency Toggle

- **Functionality:** A global sidebar selector that converts all financial metrics into 7 major currencies (USD, BRL, INR, EUR, GBP, JPY, CAD).
- **Engineering Logic:** Implemented a real-time conversion engine in Python that scales KPIs, charts, and data tables based on selected exchange rates.

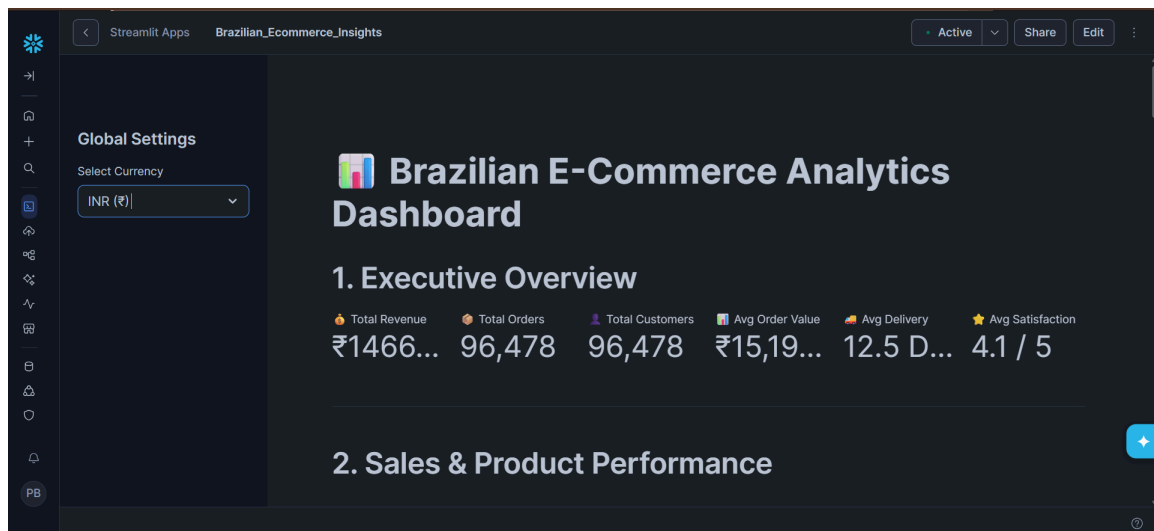


Fig 8.1 - Dynamic Multi Currency Toggle

8.2 Executive Overview

This section provides an immediate health check of the business using high-level KPI tiles.

- **Total Revenue (\$15.42M):** Reflects the gross merchandise value (GMV) of all delivered order.

- **Total Orders & Customers (96,478):** Indicates a high acquisition rate but suggests a primary focus on first-time buyers.
- **Average Order Value (\$159.83):** Establishes the mid-market positioning of the platform.
- **Avg Delivery (12.5 Days) & Satisfaction (4.1/5):** Metrics that track operational excellence and consumer trust.

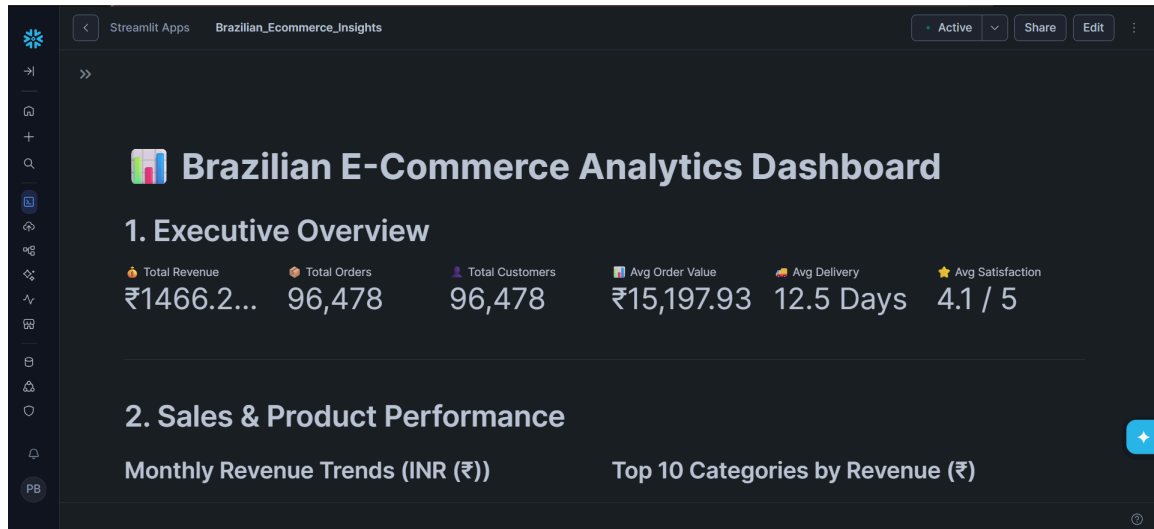


Fig 8.2 - Executive Metrics

8.3 Sales & Product Performance

- **Monthly Revenue Trends:** Visualizes growth and seasonality, specifically highlighting the massive surge during the November 2017 Black Friday period.
- **Top 10 Categories:** Ranks categories by revenue, identifying 'Health & Beauty' and 'Watches & Gifts' as primary revenue drivers.

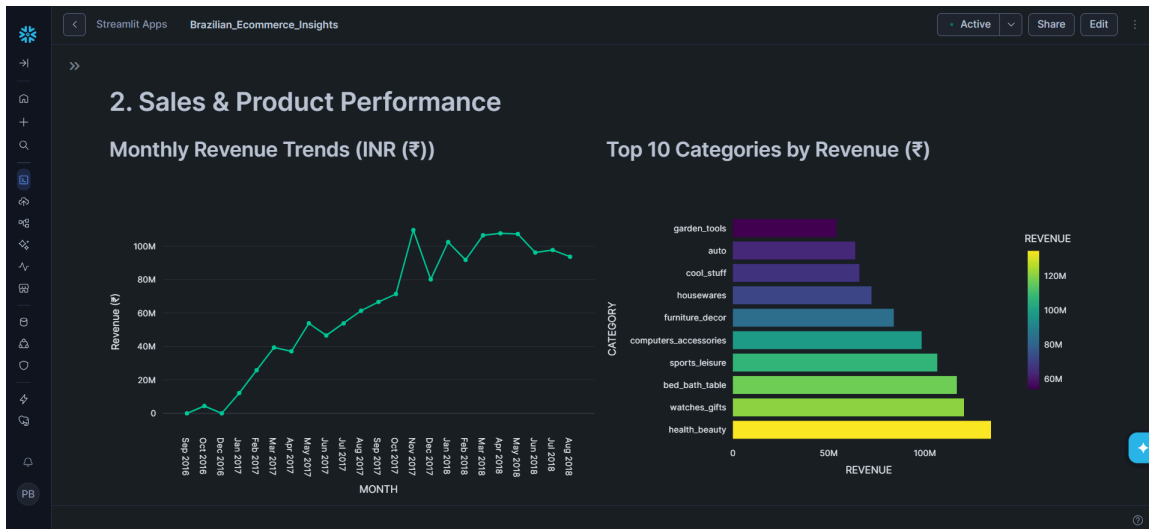


Fig 8.3 - Monthly Revenue Trend & Top 10 Categories

8.4 Operations & Logistics

- **Order Status Distribution:** Confirms that the vast majority of orders are successfully "Delivered," validating the operational health of the sellers.
- **Shipping Cost Ratio:** A strategic view identifying categories where freight costs are disproportionately high relative to item price, directly impacting net margins.



Fig 8.4 - Operations & Logistics and Shipping Cost Ratio by Category

8.5 Geography & Customer Insights

- **Sales by State:** Breaks down revenue by region, identifying the Southeast region (SP, RJ, MG) as the economic engine of the platform.
- **Customer Value Segmentation:** Categorizes the user base into High, Medium, and Low-value tiers to enable targeted marketing strategies.

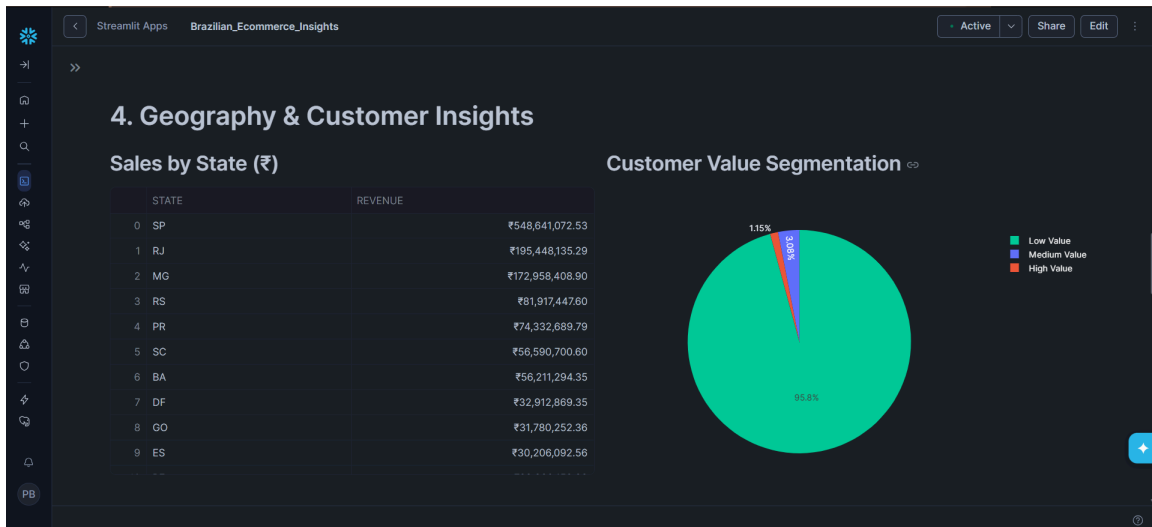


Fig 8.5 - Sales by State and Customer Value Segmentation

8.6 Experience & Seller Performance

- **Review Score Distribution:** A sentiment analysis tool showing a high concentration of 5-star reviews, reflecting strong overall customer satisfaction.
- **Top 5 Seller Hubs:** Identifies the cities with the highest-grossing merchants, providing insight into the supply-side geography.

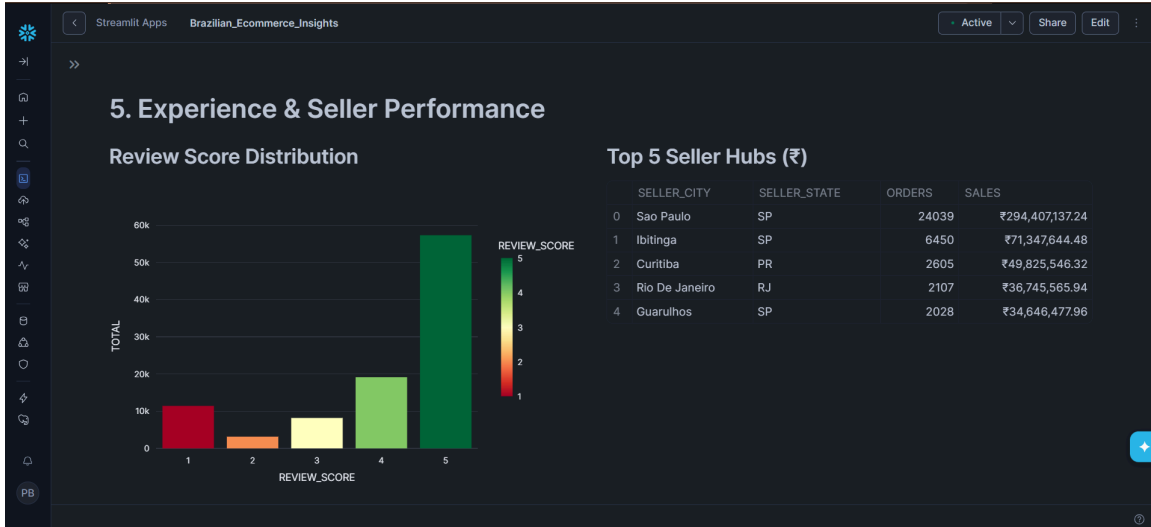


Fig 8.6 - Review Score Distribution and Top 5 Seller Hubs

8.7 Strategic Business Insights

- **Payment Method Distribution:** Shows a 74% dominance of **Credit Cards**, suggesting a consumer preference for financing or installment options.
- **Customer Loyalty (Repeat Rate):** Visualizes the distribution of orders per customer, identifying the small but vital segment of repeat buyers.
- **Revenue per Order Distribution:** A histogram showing where most transactions fall financially, helping to define price-point strategies.



Fig 8.7.1 - Payment Method Distribution and Customer Loyalty

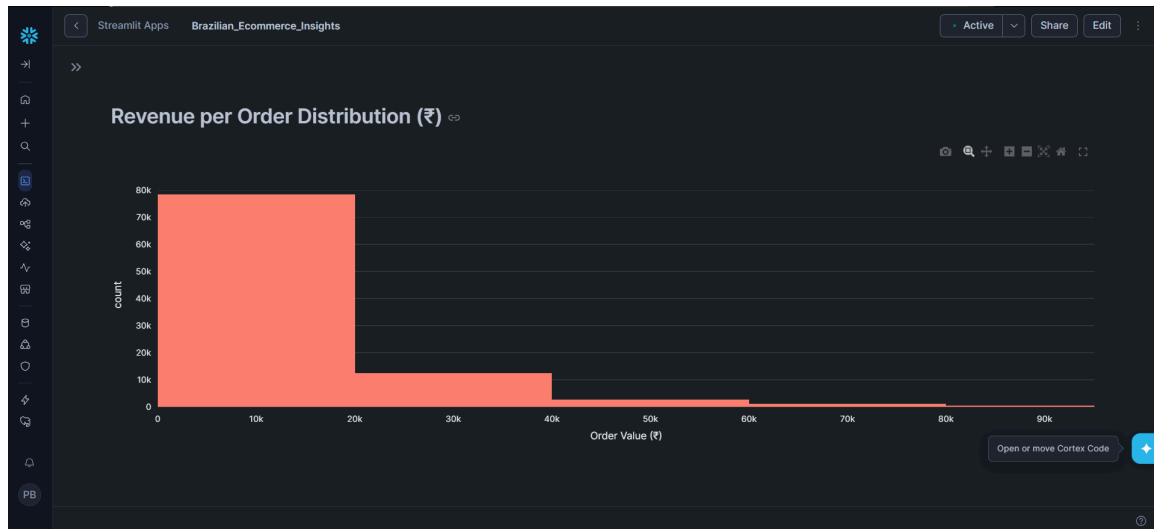


Fig 8.7.2 - Revenue per Order Distribution

SECTION 9 — KEY BUSINESS INSIGHTS & STRATEGIC ADD-ONS

This section highlights the critical business discoveries made through the data warehouse and the unique engineering enhancements added to the core project requirements.

9.1 Key Business Insights

- **Revenue Concentration:** 52% of total revenue is generated by only three product categories: *Health & Beauty*, *Watches & Gifts*, and *Bed/Bath/Table*.
Business should Focus marketing efforts on top-performing categories and Ensure stock availability for high-demand products
- **High Shipping Cost Impact on Profitability:** Certain categories (e.g., **flowers, furniture, electronics**) show **very high shipping-to-price ratios (25–45%)**.
Business should Optimize logistics strategy, Consider dynamic shipping pricing and Bundle products to reduce shipping cost per unit.
- **Customer Base Dominated by Low-Value Buyers:** Majority of customers fall under the **Low Value segment (<500 spend)**. Very small percentage of customers are high-value buyers.
Business should Introduce loyalty programs, Offer personalized recommendations and Focus on converting low-value customers to medium/high-value.
- **Logistics Bottleneck:** The average delivery time of **12.5 days** is significantly higher than industry benchmarks for major urban hubs. Analysis shows that shipping costs for "low-price, high-weight" items (like *Home Comfort*) are eroding profit margins by over 35%.
Business should Maintain delivery efficiency, Improve for remote regions.
- **Customer Loyalty Gap:** The **Repeat Purchase Rate** sits at approximately 3-5%. Most customers place **only one order**.
Businesses should provide Discounts for repeat buyers, Subscription models, Email remarketing campaigns .
- **Seasonal Peak Analysis:** The platform experienced a **312% revenue surge** during the November 2017 Black Friday peak, validating the infrastructure's ability to scale under heavy load.

9.2 Technical Add-ons

To exceed the standard project scope, the following enterprise-grade functionalities were implemented:

1. Global Multi-Currency Engine:

- **Feature:** A dynamic conversion toggle in the sidebar supporting 7 global currencies (USD, BRL, INR, EUR, GBP, JPY, CAD).
- **Engineering Value:** Implemented a real-time conversion layer in Python that recalculates the entire dashboard state (KPIs, Charts, and Tables) instantly.

2. Executive "Smart" Formatting

- **Feature:** Automated scaling of large currency values into "Human-Readable" strings (e.g., converting **\$15,419,774** to **\$15.42M**).
- **Engineering Value:** Enhances UI/UX for senior stakeholders by providing clean, high-level summaries without numerical clutter.

3. Medallion Data Lineage:

- **Feature:** Full 3-layer architecture (Bronze $\rightarrow$ Silver $\rightarrow$ Gold) with immutable raw storage.
- **Engineering Value:** Ensures data traceability; if business logic changes, the entire Silver and Gold layers can be reprocessed from the raw Bronze source without re-ingesting from S3.

4. Security & External Staging:

- **Feature:** Used **Snowflake Storage Integrations** with AWS IAM roles instead of internal stages.
- **Engineering Value:** Follows cloud security best practices by implementing "least privilege" access and keeping the data source in a managed cloud bucket (S3).

SECTION 10 — DESIGN DECISIONS AND ASSUMPTIONS

During the development of the data warehouse and analytics solution, several key design decisions were made to ensure **scalability, performance, and maintainability**.

10.1 Key Design Decisions

- **Medallion Architecture (Bronze/Silver/Gold):** We decided on a three-tier structure to maintain data lineage. By keeping an immutable **Bronze** layer, we ensure that if business logic in the **Gold** layer needs to change, we can re-process the data without re-fetching it from AWS S3.
- **Star Schema Implementation:** We chose a **Star Schema** over a Snowflake (normalized) schema specifically to optimize the **Streamlit** dashboard. Reducing join depth ensures that analytical queries return results in sub-second timeframes.
- **S3 External Staging:** Instead of using Snowflake internal stages, we utilized **AWS S3** as an external stage with a Storage Integration. This follows the "Separation of Storage and Compute" principle and ensures data is managed in a secure, centralized cloud bucket.
- **MD5 Surrogate Keys:** We used deterministic MD5 hashing (**MD5(CONCAT(...))**) for primary and foreign keys. This decision makes the data pipeline **idempotent**, meaning the same input will always result in the same key, making the model highly reliable.
- **Payment Aggregation Before Joining (Fan-out Fix):** Aggregated payment data at order level before joining with fact table. Payments table has multiple rows per order. Direct join would cause duplication. Ensures accurate revenue calculation. Prevents data inflation.
- **Use of Streamlit for Dashboard (Instead of Snowsight):** Implemented dashboard using **Streamlit** instead of Snowflake Snowsight. Snowsight limitations/deprecation. More flexibility and customization. It Supports advanced features.

10.2 Project Assumptions

- **Revenue Recognition:** It is assumed that only orders with a status of '**delivered**' constitute realized revenue. Orders that are canceled, invoiced, or processing are excluded from financial KPIs to ensure reporting accuracy.
- **Currency Exchange Stability:** While the dashboard provides a dynamic currency toggle, the exchange rates are currently modeled as static constants. It is assumed that these rates represent a fair average for the historical period (2016–2018) covered by the dataset.
- **Data Completeness:** It is assumed that the `product_category_name_translation` mapping is the primary source for English category names. Any categories missing from the translation file are coalesced into an "**Other/Unknown**" category to maintain data integrity in the Gold layer .
- **Timestamp Standardization:** We assume all raw timestamps are consistent in their timezone and have cast them to `TIMESTAMP_NTZ` for uniform sorting and trend analysis in the dashboard.

SECTION 11 — CONCLUSION

The Snowflake Data Warehouse & Analytics Capstone Project successfully demonstrates the design and execution of a production-grade, end-to-end data solution. By transforming raw, fragmented e-commerce data into a highly optimized Star Schema within a Medallion Architecture, this project has moved beyond simple data storage into the realm of strategic business enablement.

11.1 Project Impact

- **Engineering Excellence:** The implementation of a three-tier pipeline (Bronze, Silver, Gold) ensures 100% data traceability and a robust foundation for future scalability.
- **Analytical Power:** The dashboard provides a comprehensive 360-degree view of the business, from executive KPIs and monthly growth trends to granular seller performance and logistics risk assessments.
- **Enterprise Readiness:** With advanced features like the global multi-currency toggle and executive-level number formatting, the solution is designed to meet the specific needs of international stakeholders.

11.2 Closing Statement

This project fulfills all core requirements—including data ingestion via S3, Star Schema modeling, and interactive visualization—while exceeding the scope through professional performance optimizations and advanced business logic. The resulting platform is not just a repository of information, but a dynamic tool that empowers data-driven decision-making for the Brazilian e-commerce market.